Mudcard

- **what's the difference between the random subset of features vs the random subset of points that each tree in a random forest is trained on?**
  - It tabular data, the features are the columns and the points or samples are the rows.
  - So a tree can learn on a random subset of columns/features or a random subset of rows/samples.
- **What is the difference between depth and the number of features we put into the tree?**
  - They are quite different
  - Depth controls how many decisions (nodes) a tree can have
  - The number of features controls how many input variables a tree can use when making decisions.
  - The two numbers are not the same.
  - Most notably, you might have one continuous feature and keep splitting based on that one feature to build a tree or arbitrary depth.
  - If you have one-hot encoded (binary 0-1) features only in your dataset, the number of features and the depth are more closely related because once you split on a binary feature, you cannot reuse it.

Lecture 15: Tree-based ensemble methods

This lecture was prepared and first presented by Asher Labovich (DATA1030 HTA in the Fall 2025 term).

By the end of this lecture, you will be able to

- explain the difference between decision trees, random forests, and gradient boosted decision trees
- know the pros and cons for each algorithm and when is best to use each
- train these models in an ML pipeline

In short: I want you to understand tree ensemble methods as if you had discovered them yourself.

What do we want in a machine learning algorithm?

Some key factors include:

1. Fast training + prediction

2. Interpretable
3. Universal (represent any function!)
4. Has good extrapolation to data points outside the training set
5. Accurate (on the problems we care about)!!

Recap: Decision Trees

Last lecture, we talked about decision trees. They have many of the previously-mentioned properties:

1. Quick to train (even faster than linear regression under some training mechanisms!)
2. Interpretable (it's a simple rule!)
3. Universal (with infinite branches, any function can be represented to any degree of accuracy)
4. Does not go off into infinity (like polynomials do)

These are all great! However, there's one flaw:

Decision trees are **sensitive to data perturbations, very jumpy, and stepwise:** as a whole, bad at predicting the nice functions* we care about. We can see that below:

* The intuition behind this is that most "real-world" functions are differentiable / smooth, and stepwise functions are not.

 Drawing

What can we do about this? Do we just give up on decision trees?

The properties we discussed are **really** useful. If only we could find some way to augment these models in such a way that retains many of these properties but removes the jumpyness? It turns out we can!

Method 1: Random Forest

- Consider an archer with high variance and low bias
- Individual shots tend to miss, but the **average** is a good estimation

 Archer

We can treat each individual tree (f_i, a *function* over $\mathcal{X}$) as a "shot from the archer" -- high variance, relatively low bias. Like the archer, we can take $\hat{f} = \frac1N \sum_{i=1}^N f_i$ and use that as our prediction instead.

However, these "shots from the archer" have one key property: they are *uncorrelated*. If I just trained the same decision tree N times, they would be perfectly correlated. So, we want to create **uncorrelated** trees.

That's what a random forest is! Two methods:

1. Bootstrapping -- given dataset $\mathcal{D} = \{(\mathbf{x_i}, y_i)_1^m\}$, each tree T works with $\mathcal{D}_T$ sampled randomly **with replacement** from D (same size).
2. Feature subsampling -- at each split, only consider a random subset of features

The hope is, as we create more uncorrelated trees, our predictions get better and better.

 Drawing

In [1]:
```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error
import pandas as pd

rng = np.random.RandomState(42)
X_1 = rng.uniform(0, 10, size=500)
X_2 = rng.uniform(0, 10, size=500)
X = np.c_[X_1, X_2]
true_fun = lambda X: np.sin(X[:, 0]) + 3 * X[:, 1] + 1
y = true_fun(X) + rng.randn(500)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, rar

num_trees = [1, 2, 5, 10, 100, 1000]
manual_errors = []
sklearn_errors = []
for n in num_trees:
    seed_rng = np.random.RandomState(n)
    trees = []
    #Manual creation of trees
    for seed in seed_rng.randint(np.iinfo(np.int32).max, size=n):
        idx = np.random.RandomState(seed).choice(X_train.shape[0], size=X_tr
        tree = DecisionTreeRegressor(max_depth=4)
        X_boot = X_train[idx]
        y_boot = y_train[idx]
        tree.fit(X_boot, y_boot)
        trees.append(tree)

    y_pred = np.mean([tree.predict(X_test) for tree in trees], axis=0)
    manual_errors.append(mean_squared_error(y_test, y_pred))

    #sklearn random forest
    rf = RandomForestRegressor(n_estimators=n, max_depth=4, random_state=n)
    rf.fit(X_train, y_train)
    y_pred = rf.predict(X_test)
```

```
    sklearn_errors.append(mean_squared_error(y_test, y_pred))

results = pd.DataFrame({
    'Number of Trees': num_trees,
    'Manual RF Error': manual_errors,
    'sklearn RF Error': sklearn_errors
})
print(results.to_string(index=False))
```

```
Number of Trees  Manual RF Error  sklearn RF Error
              1         1.732055          1.732055
              2         1.654801          1.654801
              5         1.602447          1.602447
             10         1.577136          1.577136
            100         1.549825          1.549825
           1000         1.533020          1.532936
```

You'll notice here that the model gets consistently better as more trees are added. This is an interesting property of random forests: adding more trees **always** reduces loss ^{1}. Intuitively, this is because each tree has no direct causal effect on any other tree, and thus adding more trees is like "throwing more arrows".

^{1} In a distributional sense. If the tree added is poor by random chance the overall predictions can be worse, but on average more trees are better.

Common random forest parameters and ranges

Name	Purpose	Typical Range	Correlation with overfitting ^{2}
n_estimators	How many trees to train	≥ 100	None/Negative
max_depth	Maximum depth of trees	[1, 3, 5, 10, 30, ...]	Positive
max_features	Features to consider at each split	[0.1, 0.3, 0.5, 0.7, 0.9]	Positive

^{2} a positive correlation means that increasing this hyperparameter increases the chance of overfitting (mathematically, increases variance)

Quiz 1

Describe a positive and negative of using random forests in comparison to a single decision tree.

When might random forests fail?

Imagine we're trying to predict a house's price given a ton of features, some of which are highly correlated and some weakly correlated. **Each** tree tries to predict the entire function by itself -- so even with feature subsampling, it's possible that every tree has a

few super important features and then only uses those. It could totally miss the small, but *still relevant* correlations of the other columns! We want to fix that.

We can do this in an iterative fashion:

1. Predict with trees as normal
2. See how "far off" our predictions were from each point
3. See which points need "more focus"
4. Train new trees to predict those points more effectively
5. Repeat

This is the goal of **Gradient Boosted Decision Trees** (GBDTs) -- the most popular of which is called XGBoost (extreme gradient boosting).

For MSE regression, gradient boosting looks like this:

1. Begin with an initial mean prediction, $\hat{y}_i^0 = \mu$ for all i.
2. Calculate $y_i^t = y_i - \hat{y}_i^{t - 1}$ (the **residual** between the true value and our previous predicted value)^1
3. Train a tree f^t to predict y_i^t given X.
4. Set $\hat{y}_i^t = \hat{y}_i^{t - 1} + \nu f^t(x_i)$

(ν is the **learning rate**, a hyperparameter in GBDTs)

^{1} the residual formula for other losses is beyond the scope of this class, but can be found below in the proofs section. Interestingly, the more general formula allows for GBDTs to work with any twice-differentiable loss function, instead of RFs which only work for a few well-defined ones. Not necessary to know for this class, but interesting nonetheless!

Log-loss residual is similar, except it works in logit space (predicting z_i so that $p_i = \frac{1}{1 + e^{-z_i}}$) and works with $y_i =$ 0 or 1.



Common XGBoost parameters and ranges

Name	Purpose	Typical Range
n_estimators	How many trees to train	$[10, 20, 50, 100, 200, 500, ...]^1$
max_depth	Maximum depth of trees	$[1, 3, 5, 10, 30, ...]$
learning_rate	Effect of each additional tree	$0.01-0.1$
alpha / lambda	L1 / L2 reg on leaves^2	np.logspace(-2,2,5)
subsample / colsample_bytree	Rows chosen for each tree, columns chosen for each tree	$[0.3, 0.5, 0.7, 0.9]$

^{1} many practitioners use "early stopping" (i.e. test val loss after each tree, stop when it stops getting better) rather than tuning n_estimators.

^{2} it is beyond the scope of this class to understand how regularization in GBDTs works from a mathematical perspective. However, it is still quite useful to use in real-world training scenarios.

Early stopping with XGBoost

In [2]:
```
# Example prediction pipeline with XGBoost
from sklearn.datasets import make_regression
import xgboost

rng = np.random.RandomState(1)
X = rng.uniform(0, 10, size=100)
y = np.sin(X) + 3 * X + 1 + rng.randn(100)
X = X.reshape(-1, 1)

X_other, X_test, y_other, y_test = train_test_split(X, y, test_size=0.2, rar
X_train, X_val, y_train, y_val = train_test_split(X_other, y_other, test_siz

#--- Manually deciding how many trees to use ---
num_trees = [1, 2, 5, 10, 100, 1000, 10000]
train_errors = []
val_errors = []
for n in num_trees:
    xgb = xgboost.XGBRegressor(n_estimators=n, max_depth=3, learning_rate=0.
    xgb.fit(X_train, y_train)
    train_errors.append(mean_squared_error(y_train, xgb.predict(X_train)))
    val_errors.append(mean_squared_error(y_val, xgb.predict(X_val)))

best_tree_count = num_trees[np.argmin(val_errors)]
xgb = xgboost.XGBRegressor(n_estimators=best_tree_count, max_depth=3, learni
xgb.fit(X_other, y_other)
y_pred = xgb.predict(X_test)
test_error = mean_squared_error(y_test, y_pred)

print('--- Manually tuning number of trees ---')
results = pd.DataFrame({
    'Number of Trees': num_trees,
    'Train Error': train_errors,
    'Val Error': val_errors
})

print(results.to_string(index=False))
print(f"With {best_tree_count} trees, we get a test error of {test_error}")

print('--- Running with early stopping ---')
eval_set = [(X_val, y_val)]
xgb = xgboost.XGBRegressor(max_depth=3, learning_rate=0.1, random_state=42,
xgb.fit(X=X_train, y=y_train, eval_set=eval_set, verbose=False)
best_num_trees = xgb.best_iteration

xgb = xgboost.XGBRegressor(n_estimators=best_num_trees, max_depth=3, learnir
xgb.fit(X_other, y_other)
y_pred = xgb.predict(X_test)
```

```
test_error = mean_squared_error(y_test, y_pred)
print(f"With {best_num_trees} trees, we get a test error of {test_error}")
```

```
--- Manually tuning number of trees ---
 Number of Trees  Train Error  Val Error
               1    63.246550  68.502103
               2    52.549350  58.190349
               5    30.273585  35.046523
              10    12.396203  15.059507
             100     0.233589   1.361979
            1000     0.000012   2.167581
           10000     0.000011   2.167693
With 100 trees, we get a test error of 0.7171087769181819
--- Running with early stopping ---
With 59 trees, we get a test error of 0.6971942835279534
```

Quiz 2

In your own words, what is the intuitive difference between decision trees, random forests, and GBDTs?

Special Properties of Decision Trees

Natively handling missing values

Most ML models cannot naturally handle missing values because mathematical operations on unknown values (like multiplying NA by a number) aren't defined. There are all sorts of methods to deal with missing values for those models, some of which we'll explain later.

However, when working with tree-based models, this is **unnecessary**. When deciding on the usefulness of a split-point (e.g. $X_3 \geq 1$), tree models (RF in sklearn and GBDTs in XGBoost/other packages) consider two possibilities, where all missing values go left or all go right. Then, it chooses to push all missing values in the direction that would decrease loss the most.

This way, tree-based models actually **gain** information from missing values (as we'd like them to)! No need to preprocess missing values when working with tree-based models; leave them in.

 Drawing

Monotonic constraints

A lot of the time you might know that a variable in your dataset is monotonic. For example, sugar probably has a positive relationship with "is a dessert". Linear models (linreg, logreg) **only** have monotonic relationships, so they figure it out automatically.

However, since tree-based models look for non-linear complex relationships, they can sometimes miss these types of relationships! If we have prior knowledge, we can tell the model that some features are monotonic. This helps with generalization and interpretability. **Understand your dataset! Run EDA! Prior knowledge helps!**

The most basic and commonly used monotonic constraint algorithm just blocks any split-points on a monotonic feature that would result in non-monotonic relationships.

 dessert

In [3]:
```
#Implementation and comparison of monotonic constraints

import numpy as np
from sklearn.model_selection import train_test_split, KFold, GridSearchCV
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error
import pandas as pd

rng = np.random.RandomState(42)
X_1 = rng.uniform(0, 10, size=500)
X_2 = rng.uniform(0, 10, size=500)
y = np.sin(X_1) + 3 * X_2 + 1 + rng.randn(500) #Monotonic relationship for X
X = np.c_[X_1, X_2]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, rar

xgb_params = {'n_estimators': [10, 20, 50, 100, 200], 'max_depth': [2, 3, 5,

#Basic XGBoost, no monotonic constraints
xgb_basic = xgboost.XGBRegressor(random_state=42)
xgb_basic_cv = GridSearchCV(xgb_basic, xgb_params, cv=KFold(n_splits=5, shuf
xgb_basic_cv.fit(X_train, y_train)
train_error_basic = mean_squared_error(y_train, xgb_basic_cv.predict(X_trair
test_error_basic = mean_squared_error(y_test, xgb_basic_cv.predict(X_test))

#XGBoost with monotonic constraints
xgb_motonic = xgboost.XGBRegressor(monotone_constraints=(0, 1), random_state
xgb_motonic_cv = GridSearchCV(xgb_motonic, xgb_params, cv=KFold(n_splits=5,
xgb_motonic_cv.fit(X_train, y_train)
train_error_monotonic = mean_squared_error(y_train, xgb_motonic_cv.predict(X
test_error_monotonic = mean_squared_error(y_test, xgb_motonic_cv.predict(X_t

df = pd.DataFrame({
    'Model': ['Basic', 'Monotonic'],
    'Train Error': [train_error_basic, train_error_monotonic],
    'Test Error': [test_error_basic, test_error_monotonic]
})
print(df.to_string(index=False))
```

```
   Model  Train Error  Test Error
   Basic     0.615150    1.013988
Monotonic     0.651914    1.000964
```

Mudcard

Deeper math behind RF / GBDTs (NOT necessary, but here for those who want it!)

Random Forests (mostly):

These formulas describe the effect of m, the number of trees, on the bias and variance of a random forest.

Bias of random forest: $$\begin{align*} B\left[\frac1m \sum_{i=1}^m f_i\right] &= \\ \mathbb{E}\left[\frac1m \sum_{i=1}^m f_i - f\right] &= \\ \mathbb{E}\left[\frac1m \sum_{i=1}^m (f_i - f)\right] &= \\ \frac1m \sum_{i=1}^m B[f_i] &= \\ B[f_i] \end{align*}$$

Thus, the number of trees has **no** effect on bias. (This is why practitioners often use higher max depth for random forests, since that reduces the bias of each tree, and thus the floor error of the ensemble)

Variance of random forest: $$\begin{align*} \operatorname{Var\left(\frac1m \sum_{i=1}^m f_i\right)} &= \\ \frac{1}{m^2} \left(\sum_{i=1}^m Var(f_i) + \sum_{i \neq j} 2\operatorname{Cov(f_i, f_j)} \right) &= \\ \frac{1}{m^2} \left(m \operatorname{Var}(f_i) + \rho m(m-1) \operatorname{Var}(f_i)\right) &= \\ \frac{\operatorname{Var}(f_i) + \rho (m - 1) \operatorname{Var}(f_i)}{m} &= \\ \operatorname{Var}(f_i) \left(\rho + \frac{1 - \rho}{m}\right) \end{align*}$$

Here we define ρ to mean the pairwise correlation between trees. You might be wondering what this value represents, and how we might go about estimating it. My recommendation is to just think of it a value from 0-1 that becomes smaller as you "increase randomness" (i.e. by reducing features sampled at each node). The less-handwavy math is surprisingly complex and involves **way** more notation than I'd like to include here (email me if you want to go more in depth).

However, there are some things to note about this formula:

1. It creates a "variance floor" of $\sigma^2 \rho$, which means that random forests will **always** have some level of variance.
2. It "proves" our earlier statement on how adding more trees always helps (in a distributional sense -- if the tree you added was awful, it might make it worse on the specific dataset).

GBDTs:

Earlier I described GBDTs as simply "predicting the difference between the true value and the predicted value". While that's true for MSE, that's not quite the whole story. In fact, under the hood, GBDTs are using a second-order Taylor approximation that just *happens* to be nice for the examples we care about (in particular, MSE and log-loss).

To understand why we've got to think about what it *means* to iteratively improve upon previous models.

We have an existing model $f^t(x)$ with loss $L(f^t, y)$ and we wish to add on another model -- call it $p(x)$, so that $L(f^t + p, y)$ is minimized. Since ν is small, νp is also small, so we can *reasonably* assume a second-order Taylor approximation on p since we know we'll later apply ν.

$$\begin{align*} L(f^t + p, y) &= \\ \sum_{i=1}^n L(f^t(x_i) + p(x_i), y_i) &\approx \\ \sum_{i=1}^n L(f^t(x_i), y_i) + \frac{\partial L(f^t(x_i), y_i)}{\partial f^t(x_i)} p(x_i) + \frac{\partial^2 L(f^t(x_i), y_i)}{\partial f^t(x_i)^2} \frac{(p_i(x))^2}{2} \end{align*}$$

Now of course we want to minimize this with respect to p -- so the first term doesn't matter, only the latter two do. For simplicity we call $g_i = \frac{\partial L(f^t(x_i), y_i)}{\partial f^t(x_i)}$ and $h_i = \frac{\partial^2 L(f^t(x_i), y_i)}{\partial f^t(x_i)^2}$ for the gradient and hessian, respectively.

From calculus we know that this is minimized when the derivative w/r/t $p(x_i)$ for all i is 0, so we take that derivative:

$$\begin{align*} g_i + h_i p(x_i) = 0 &\rightarrow \\ p(x_i) = - \frac{g_i}{h_i} \end{align*}$$

This is the complete general formula for a gradient-boosting machine. If you want 90% of the understanding of GBDTs, stop here. We simply use p as the pseudo-residual we're predicting at a given step and keep updating step-by-step. We can sanity-check this existing math on MSE -- $L(x, y) = \frac12 (y - x)^2$, for example, we have that $g_i = x_i - y_i$ and $h_i = 1$, so $p_i = y_i - x_i$, as earlier.

If you **truly** want the full story -- specifically, how this works with **decision trees**, here it is:

XGBoost and similar packages work on more than just MSE, they work on ANY twice-differentiable function. Calculating the p_i for these is easy -- but then how do you train a tree to predict the p_i, if we're working on a different loss function than simple MSE? Regression trees are designed to work with MSE, after all.

To do so, we assume we already have a tree and we're deciding what the split point should be. We know that this leaf value must be constant, so we can redo the earlier math for a specific leaf, but set $p(x_i) = p$ for all i in that leaf. I won't do it here but you can fairly easily verify that you get $p = - \frac{\sum_{i=1}^n g_i}{\sum_{i=1}^n h_i}$. We

can simplify this more by calling $G = \sum_{i=1}^n g_i$ and $H = \sum_{i=1}^n h_i$, so $p = -\frac{G}{H}$.

Great! So, we've figured out what to put *in* the leaf, but we still don't know how to *split* the existing tree. We can simply plug this back into the original formula for L and we get:

$$\begin{align*} \sum_{i=1}^n L(f^t(x_i) + p, y_i) &= \\ \sum_{i=1}^n L_i^{t-1} + -g_i\frac{G}{H} + h_i \frac{G^2}{2H^2} &= \\ L^{t-1} - \frac{G^2}{H} + \frac{G^2}{2H} &= \\ L^{t-1} + \frac{G^2}{2H} \end{align*}$$

When we choose a split point, we're turning one leaf into two. So we can calculate the difference in loss over the two leaves versus the one by letting L and R be the left and right leaves, respectively. $$L_T - (L_R + L_L) = L^{t-1} + \frac{G_T^2}{2H_T} - \left(L_L^{t-1} + \frac{G_L^2}{2H_L} + L_R^{t-1} + \frac{G_R^2}{2H_R}\right)$$

Since the previous loss doesn't change, we simply choose the split-point where $$\frac{G_T^2}{2H_T} - \frac{G_L^2}{2H_L} - \frac{G_R^2}{2H_R}$$ is maximized.

And... we're done! That's how gradient-boosting decision trees are trained, from scratch.